Gradient Descent

Method 2: Be Clever

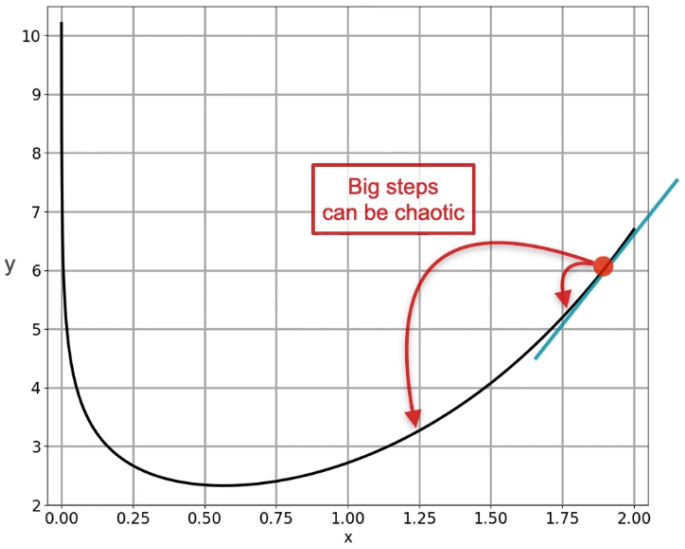
Try something smarter…

new point = old point - slope

$$x_1 = x_0 - f'(x_0)$$

$$x_1 = x_0 - \alpha f'(x_0)$$

Learning rate



Steps

Function: $f(x)$ Goal: find minimum of $f(x)$

Step 1:

Define a learning rate α

Choose a starting point x_0

Step 2:

Update: $x_k = x_{k-1} - af'(x_{k-1})$

Step 3:

Repeat Step 2 until you are close enough to the true minimum x^*

$f(x) = e^x - \log(x)$ $f'(x) = e^x - \frac{1}{x}$

Start: $x = 0.05$ Rate: $\alpha = 0.005$

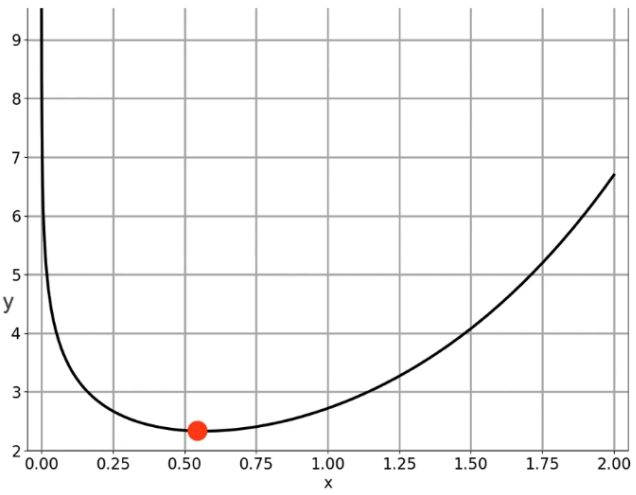
Find:
$f'(0.05) = -18.9$

Move by $-0.005 f'(0.05)$ $x \mapsto 0.1447$

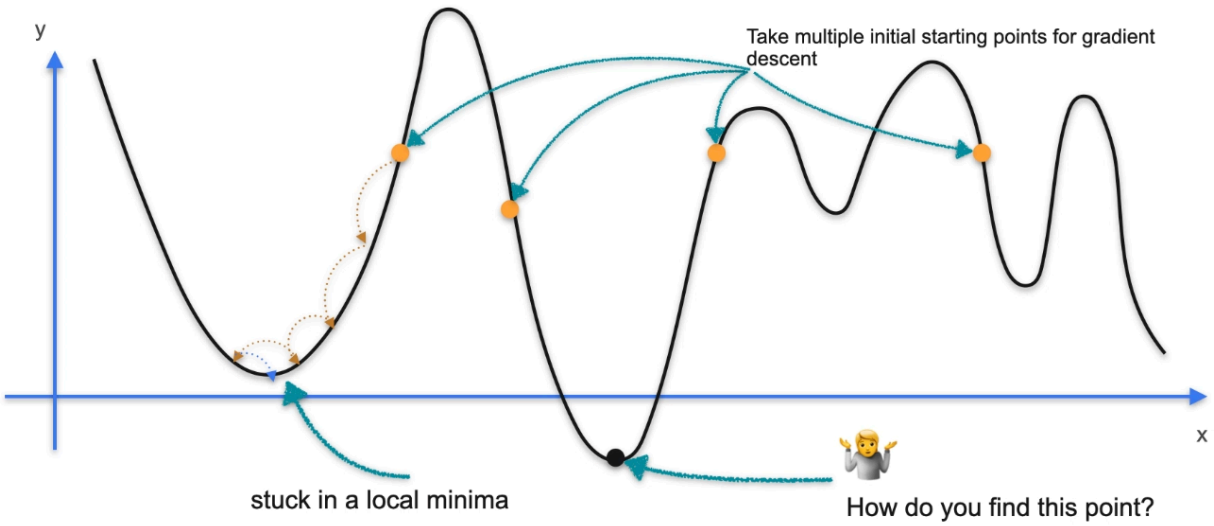
Find:
$f'(0.1447) = -5.7552$

Move by $-0.005 f'(0.1447)$ $x \mapsto 0.1735$

Repeat!



Drawbacks of Gradient Descent



Idea for the Gradient Descent

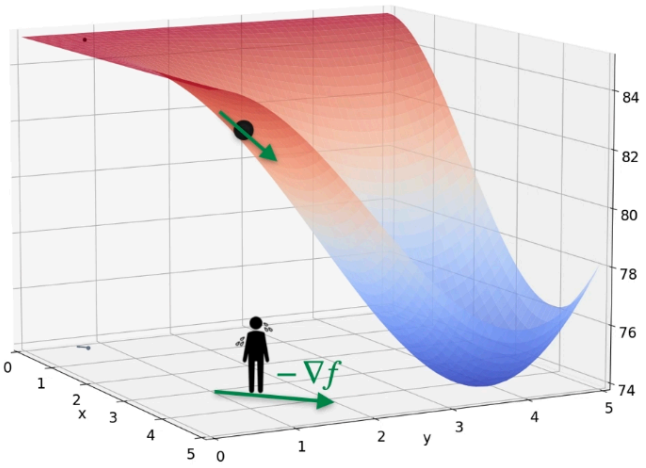
Initial position: (x_0, y_0)

Direction of greatest ascent: ∇f

Direction of greatest descent: $-\nabla f$

Updated position: $\underbrace{(x_0, y_0) - \alpha \nabla f}_{(x_1, y_1)}$

Better point!



Function: $f(x, y)$ Goal: find minimum of $f(x, y)$

Step 1:

Define a learning rate α

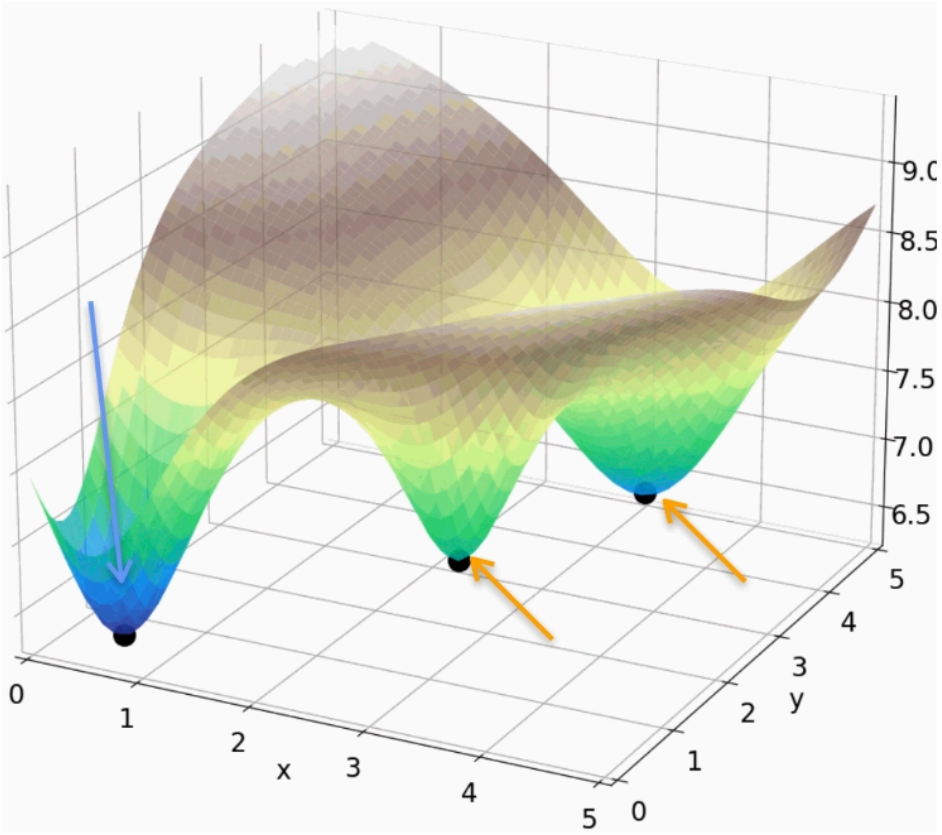
Choose a starting point (x_0, y_0)

Step 2:

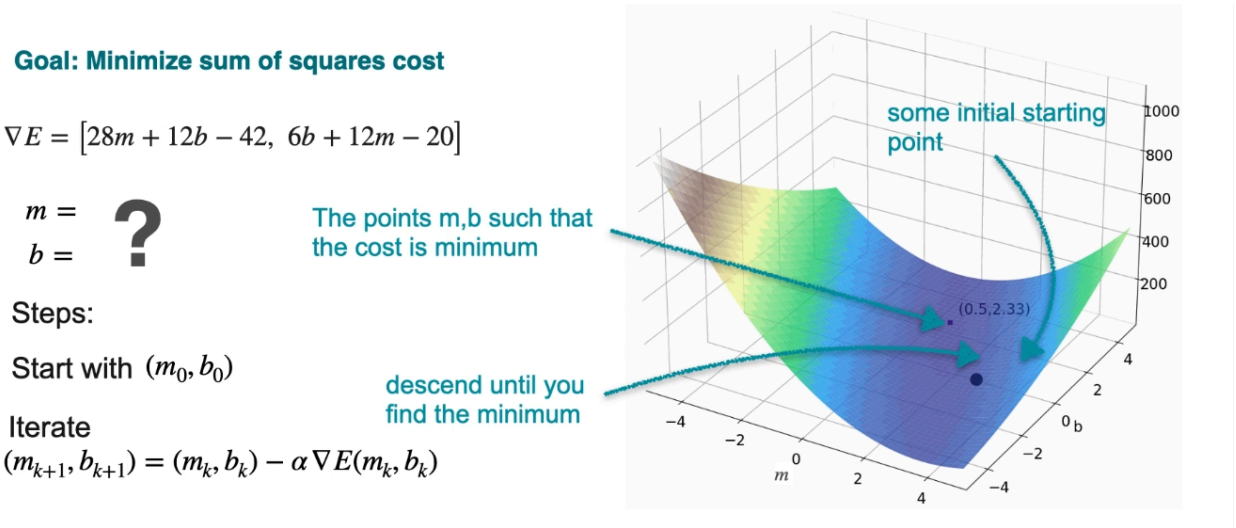
Update: $\begin{bmatrix} x_k \\ y_k \end{bmatrix} = \begin{bmatrix} x_{k-1} \\ y_{k-1} \end{bmatrix} - \alpha \nabla f(x_{k-1}, y_{k-1})$

Step 3:

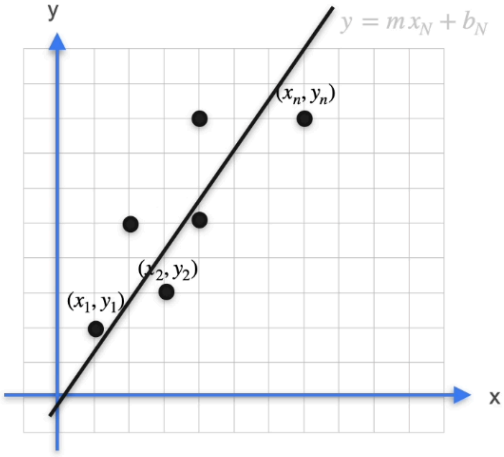
Repeat Step 2 until you are close enough to the true minimum (x^*, y^*)



To avoid the local minimum, try starting from different points.
There is no way to tell if we get to the local or global minimum, but we can try adding more iterations!



Gradient Descent



$$\mathscr{L}(m, b) = \frac{1}{2n}\left[(mx_1 + b - y_1)^2 + \cdots + (mx_n + b - y_n)^2\right]$$

$$\begin{bmatrix} m_N \\ b_N \end{bmatrix} \rightarrow \begin{bmatrix} m_N \\ b_N \end{bmatrix} = \begin{bmatrix} m_{N-1} \\ b_{N-1} \end{bmatrix} - \alpha \nabla \mathscr{L}_1(m_{N-1}, b_{N-1})$$